

AddisCoder: Week 2, Lecture 6b

Taught by Huy Nguyen. AddisCoder 2025.

While loops vs For loops

While loops and For loops are the two ways we've taught you to repeat code. Knowing when to use which is tricky.

```
while condition:
    code inside the loop
code outside the loop
```

If the condition is True, run the code in the while loop, and then start back at the beginning of the while loop.

If the condition is False, skip the code in the while loop.

```
current_step = 0 # initialize counter
while current_step < 10: # condition
    print("clap") # body
    current_step += 1 # increment counter

clap
clap
clap
clap
clap
clap
clap
clap
clap
clap
clap
```

For loops

```
for element in list:
    code inside the loop
code outside the loop
```

Example: What will print in the code below?

```
for n in [1,2,3,4,5]:
    print(n)
print("daniel")
for w in [1,2,3]:
```

```

    print(w)
print("Huy")

1
2
3
4
5
daniel
1
2
3
Huy

for i in range(200,400):
    if i % 20 == 0:
        continue
    if i % 30 == 0:
        print(i)

```

The truth about loops: You can do all of your looping with while loops if you want. There's nothing you can do only with a forloop but not a whileloop. But forloops give a cleaner way to do many common looping operations when you know ahead-of-time how much you will loop.

```

# Let's take a sample problem: summing squares from 1 up to n.
# The formula for this is 1^2 + 2^2 + 3^2 + ... + n^2

# This is solution using a for loop:
def sum_of_squares_while(n):
    total = 0
    for i in range(1, n+1):
        total += i**2
    return total

print("While sol:", sum_of_squares_while(10))

# This is the solution using a while loop:
def sum_of_squares_for(n):
    total = 0
    i = 1
    while i <= n:
        total += i**2
        i += 1
    return total

print("For sol:", sum_of_squares_for(10))

# Notice the direct transformation from for to while. (Write up on the board)

```

While sol: 385

For sol: 385

```
cities=['addis ababa', 'boston', 'paris', 'cairo']
```

```
# print all names using a for loop
```

```
for city in cities:  
    print(city)
```

```
# print all names using a while loop
```

```
i = 0  
while i < len(cities):  
    print(cities[i])  
    i += 1
```

```
addis ababa
```

```
boston
```

```
paris
```

```
cairo
```

```
addis ababa
```

```
boston
```

```
paris
```

```
cairo
```

```
# Note: `break` and `continue` work in both.
```

```
# Some problems can *only* be solved with a while loop.
```

```
# These are problems where we don't know how many iterations we need to do.
```

```
# Example: find the first power of 2 that is greater than 1000.
```

```
# We don't know how many times we need to multiply 2 by itself to get to 1000.
```

```
def first_power_of_two_greater_than(n):  
    power = 1  
    while 2**power <= n:  
        power += 1  
    return power
```

```
print(first_power_of_two_greater_than(1000))
```

```
10
```

```
# Therefore, the general rule is this:
```

```
# whenever you know how many iterations you need to do, use a for loop.
```

```
# whenever you don't know how many iterations you'll need, use a while loop.
```

Nested Loops

We can put loops inside of loops! We call this "nested loops"

```
# Nested loops are loops inside of loops.  
# They are useful for solving problems that are made up of pieces that  
are themselves made up of pieces.
```

```
# example: print a multiplication table  
# 1 2 3 4 5 6 7 8 9 10  
# 2 4 6 8 10 12 14 16 18 20  
# 3 6 9 12 15 18 21 24 27 30  
# ...
```

```
# We can do this with a nested for loop:  
for i in range(1, 11):  
    row = str(i)  
    for j in range(2, 11):  
        row = row + ", " + str(i*j)  
    print(row)
```

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10  
2, 4, 6, 8, 10, 12, 14, 16, 18, 20  
3, 6, 9, 12, 15, 18, 21, 24, 27, 30  
4, 8, 12, 16, 20, 24, 28, 32, 36, 40  
5, 10, 15, 20, 25, 30, 35, 40, 45, 50  
6, 12, 18, 24, 30, 36, 42, 48, 54, 60  
7, 14, 21, 28, 35, 42, 49, 56, 63, 70  
8, 16, 24, 32, 40, 48, 56, 64, 72, 80  
9, 18, 27, 36, 45, 54, 63, 72, 81, 90  
10, 20, 30, 40, 50, 60, 70, 80, 90, 100
```

```
# There are many more problems where nested loops are useful.  
# For example, we can use nested loops to print a triangle of stars:  
# *  
# **  
# ***
```

```
# We can do this with a nested for loop:  
for i in range(1, 4):  
    row = ""  
    for j in range(1, i+1):  
        row = row + "*"   
    print(row)
```

```
*  
**  
***
```

Note that they don't have to be just for loops, we can use while loops as well:

```
i = 1
while i <= 3:
    row = ""
    j = 1
    while j <= i:
        row = row + "*"
        j += 1
    print(row)
    i += 1
```

```
*
**
***
```

Nested lists

```
labs = [['Emran', 'Yihenew', 'Elyana'], ['Abinet', 'Kaleb'],
        ['Saron'], ['Famia']]
print(labs[0])
```

```
['Emran', 'Yihenew', 'Elyana']
```

```
i = 0
for i in range(len(labs)):
    s = 'List of students in lab '+str(i)+': '
    for student in labs[i]:
        s += student + ', '
    print(s)
```

```
List of students in lab 0: Emran, Yihenew, Elyana,
List of students in lab 1: Abinet, Kaleb,
List of students in lab 2: Saron,
List of students in lab 3: Famia,
```

Assigning Complex Types

Example one:

```
# Simple Types
```

```
x = 10
y = 20
x = y
print(x)
print(y)
```

[PythonTutor link for visualization](#)

Example two:

```
# Complex Types: everything other than int, float, bool, string

dic1 = {"jelani": 100, "heather": 95, "daniel": 90}
dic2 = dic1
dic1["binyam"] = 105

print(dic1)
print(dic2)
```

[PythonTutor link for visualization](#)

Example three:

```
def fun_func():
    x = 50
    y = 60
    num = x + y
    return

fun_func()

print(num)
```

[PythonTutor link for visualization](#)

Modules

There are millions of developers out there who write code and build things. There is common functionality that many people need that isn't built into Python natively. "Modules" using `import` statements solve this problem by letting you import functions that other people have written and use them in your own code.

```
# import using the "import" statement
import random

# Pick a random number between 0 and 10
x = random.randint(0, 10)
print(x)

7

# Let's see how the distribution looks? Is it really random?
counter = {}
for _ in range(10000):
    x = random.randint(0, 10)
    if x in counter:
        counter[x] = counter[x] + 1
    else:
```

```

        counter[x] = 1

print(counter)

{7: 896, 9: 862, 6: 919, 3: 881, 4: 884, 8: 917, 10: 951, 2: 932, 5:
897, 0: 905, 1: 956}

# Other useful functions in the random module:
# Let's play with some.
# random.random() - returns a random float between 0 and 1
# random.choice(list) - returns a random element from the list
# random.shuffle(list) - shuffles the list in place
# random.sample(list, k) - returns a list of k random elements from
the list
# random.randrange(start, stop, step) - returns a random integer from
the range
# random.uniform(a, b) - returns a random float between a and b

# Let's use one of these to make a fortune teller.
fortunes = [
    "You will be hungry again in one hour.",
    "Tomorrow will be a great day for you!",
    "Things will go well for you today.",
    "Hard work pays off in the future, laziness pays off now.",
    "Change can hurt, but it leads a path to something better.",
    "People are naturally attracted to you.",
    "You learn from your mistakes... You will learn a lot today.",
    "If you have something good in your life, don't let it go!",
    "What ever you're goal is in life, embrace it visualize it, and
for it will be yours.",
    "Your shoes will make you happy today.",
]

def tell_fortune():
    print(random.choice(fortunes))

tell_fortune()

Tomorrow will be a great day for you!

# There's another common way you'll see things imported.
# This is the "from ... import ... as ..." statement.
# This is useful if you want to use a shorter name for something. No
difference otherwise.
from random import choice

def tell_fortune():
    print(choice(fortunes))

```

```

# Math is another common package to import.
import math

# Let's try to find the square root of 2
print(math.sqrt(2))

# Other common functions in the math module:
# math.ceil(x) - returns the smallest integer greater than or equal to x
# math.floor(x) - returns the largest integer less than or equal to x
# math.pow(x, y) - returns x raised to the power y
# math.log(x) - returns the natural logarithm of x
# math.log10(x) - returns the base-10 logarithm of x
# math.pi - returns the value of pi
# math.abs(x) - returns the absolute value of x

# Out of curiosity, which is faster: math.pow() or the ** operator we
# learned in week 1?

# We can use the time module to measure the time it takes to run a
# piece of code.
import time

# time.time() returns the number of seconds that have passed since
# January 1, 1970.
start = time.time()
for i in range(100000):
    x = math.pow(2, 32)
end = time.time()
print("Math.pow: " + str(end - start))

start = time.time()
for i in range(100000):
    x = 2**32
end = time.time()

print("** operator: " + str(end - start))

```

Looks like ** is faster! 0.001 seconds is 1 millisecond.

By the way, for context, a blink of an eye is 100ms.

```

# You can use the "help" function to get more information about a
# function.
help(random.choice)

```

If you ever want to try doing something more complicated, like building a website/game, training a machine learning model, writing a WhatsApp bot, or anything else, you will definitely be using libraries for that.

Input

We've learned about how to allow your program to provide output (print to the screen). You can also provide input to your program to make it *dynamic*.

```
# Use the `input` function to take input
# See the bar that shows up at the top to provide input!

inp = input("Enter a number: ")
print(inp)

Enter a number: 75

75

# input type is always a string (raw characters)
print(type(inp))

# If you want something other than a string, you need to convert it by
"casting" to a different type.
# Here we cast to an integer
int_input = int(input("Enter a number: "))
print(int_input)

# See what happens if we provide something other than an integer
int_input = int(input("Enter a number: "))
print(int_input)
```

Slicing

```
x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
x

[1, 2, 3, 4, 5, 6, 7, 8, 9]

# x[start:end:step]
x[1:5:2]

[2, 4]

# We can drop any of (start, end, step) and they will be replaced with
the default values.
x[1::2]

[2, 4, 6, 8]

x[1:4]

[2, 3, 4]

x[:4:2]
```

```
[1, 3]
```

```
x[::]
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```